

MasterClass in DSA

Exercise Set - 4

Topic: Doubly Linked List

(Common exercises applicable to both DLL and DCLL)

Total Exercises: 30

Easy: 8 | Intermediate: 14 | Difficult: 8

*CoreCode Programming Academy
An Academy by Yogeshwar Shukla*

General Instructions

1. Extend your existing list.h and list.c implementations with these new functions.
2. Use the established type definitions: list_t, node_t, status_t, data_t, len_t.
3. Use existing constants: SUCCESS, TRUE, FALSE, LIST_DATA_NOT_FOUND, LIST_EMPTY.
4. Leverage your existing helper functions: generic_insert(), generic_delete(), search_node(), get_node().
5. These exercises have identical solutions for DLL and DCLL (with minor termination differences).
6. Test using assertions as demonstrated in use_list.c.

Already Implemented (Not Included in This Set)

The following are already in your implementation and excluded from this exercise set:

- create_list, insert_start, insert_end, insert_after, insert_before
- get_start, get_end, pop_start, pop_end
- remove_start, remove_end, remove_data, clear_list
- is_list_empty, find_in_list, get_list_length, show_list, destroy_list
- get_concatated_list, concat_lists, merge_lists
- get_reversed_list, reverse_list
- generic_insert, generic_delete, search_node, get_node

List Notation

DLL: [HEAD] ⇄ [d1] ⇄ [d2] ⇄ [d3] ⇄ [NULL]

DCLL: [HEAD] ⇄ [d1] ⇄ [d2] ⇄ [d3] ⇄ ○ (circular back to HEAD)

Section A: Easy Exercises

Exercise 1: Display Backward EASY

Write a function to display all elements from end to start (backward traversal). For DLL, first traverse to find the last node, then traverse backward using prev pointers. For DCLL, start directly from `p_list->prev`.

```
void show_list_backward(list_t* p_list, const char* msg)
```

Sample Input:

```
[HEAD]⇄[10]⇄[20]⇄[30]⇄...
```

Sample Output:

```
[END]⇄[30]⇄[20]⇄[10]⇄[START]
```

Exercise 2: Get Data at Index EASY

Write a function to retrieve the data at a given 0-based index. Return `LIST_DATA_NOT_FOUND` if index is invalid (negative or $\geq$ length).

```
status_t get_data_at_index(list_t* p_list, int index, data_t* p_data)
```

Sample Input:

```
[HEAD]⇄[10]⇄[20]⇄[30]⇄[40]⇄..., index = 2
```

Sample Output:

```
*p_data = 30, returns SUCCESS
```

Exercise 3: Insert at Position EASY

Write a function to insert a new node at a given 0-based position. Position 0 means insert at start. Return `LIST_DATA_NOT_FOUND` if position is invalid.

```
status_t insert_at_position(list_t* p_list, data_t new_data, int position)
```

Sample Input:

```
[HEAD]⇄[10]⇄[30]⇄[40]⇄..., new_data = 20, position = 1
```

Sample Output:

```
[HEAD]⇄[10]⇄[20]⇄[30]⇄[40]⇄...
```

Exercise 4: Delete at Position EASY

Write a function to delete the node at a given 0-based position. Return LIST_EMPTY if list is empty, LIST_DATA_NOT_FOUND if position is invalid.

```
status_t delete_at_position(list_t* p_list, int position)
```

Sample Input:

```
[HEAD]⇨[10]⇨[20]⇨[30]⇨[40]⇨..., position = 2
```

Sample Output:

```
[HEAD]⇨[10]⇨[20]⇨[40]⇨...
```

Exercise 5: Count Occurrences EASY

Write a function that counts how many times a specific value appears in the list.

```
len_t count_occurrences(list_t* p_list, data_t value)
```

Sample Input:

```
[HEAD]⇨[5]⇨[3]⇨[5]⇨[7]⇨[5]⇨..., value = 5
```

Sample Output:

```
3
```

Exercise 6: Get Maximum Element EASY

Write a function to find and return the maximum data value in the list. Return LIST_EMPTY if the list is empty.

```
status_t get_max(list_t* p_list, data_t* p_max)
```

Sample Input:

[HEAD]⇄[45]⇄[12]⇄[78]⇄[34]⇄...

Sample Output:

*p_max = 78, returns SUCCESS

Exercise 7: Get Minimum Element EASY

Write a function to find and return the minimum data value in the list. Return LIST_EMPTY if the list is empty.

```
status_t get_min(list_t* p_list, data_t* p_min)
```

Sample Input:

[HEAD]⇄[45]⇄[12]⇄[78]⇄[34]⇄...

Sample Output:

*p_min = 12, returns SUCCESS

Exercise 8: Check if Lists are Equal EASY

Write a function that checks if two lists contain the same elements in the same order.

```
status_t are_lists_equal(list_t* p_list_1, list_t* p_list_2)
```

Sample Input:

List1: [HEAD]⇄[1]⇄[2]⇄[3]⇄..., List2: [HEAD]⇄[1]⇄[2]⇄[3]⇄...

Sample Output:

Section B: Intermediate Exercises

Exercise 9: Insert in Sorted Order INTERMEDIATE

Write a function to insert a new element into an already sorted list (ascending order) such that the list remains sorted after insertion.

```
status_t insert_sorted(list_t* p_list, data_t new_data)
```

Sample Input:

```
[HEAD]⇄[10]⇄[20]⇄[40]⇄[50]⇄..., new_data = 30
```

Sample Output:

```
[HEAD]⇄[10]⇄[20]⇄[30]⇄[40]⇄[50]⇄...
```

Exercise 10: Remove All Duplicates INTERMEDIATE

Write a function to remove all duplicate values from an unsorted list, keeping only the first occurrence of each value. Return the count of nodes removed.

```
len_t remove_duplicates(list_t* p_list)
```

Sample Input:

```
[HEAD]⇄[5]⇄[3]⇄[5]⇄[7]⇄[3]⇄[5]⇄...
```

Sample Output:

```
[HEAD]⇄[5]⇄[3]⇄[7]⇄..., returns 3
```

Exercise 11: Replace All Occurrences INTERMEDIATE

Write a function that replaces all occurrences of `old_value` with `new_value`. Return the count of replacements made.

```
len_t replace_all(list_t* p_list, data_t old_value, data_t new_value)
```

Sample Input:

```
[HEAD]⇄[5]⇄[3]⇄[5]⇄[7]⇄..., old_value = 5, new_value = 100
```

Sample Output:

```
[HEAD]⇄[100]⇄[3]⇄[100]⇄[7]⇄..., returns 2
```

Exercise 12: Swap Two Nodes by Value INTERMEDIATE

Write a function to swap two nodes given their data values. Swap the nodes themselves (not just data), handling all prev and next pointer updates including the case when nodes are adjacent.

```
status_t swap_nodes(list_t* p_list, data_t data1, data_t data2)
```

Sample Input:

[HEAD]⇨[1]⇨[2]⇨[3]⇨[4]⇨..., data1 = 2, data2 = 4

Sample Output:

[HEAD]⇨[1]⇨[4]⇨[3]⇨[2]⇨...

Exercise 13: Swap Adjacent Pairs INTERMEDIATE

Write a function to swap every two adjacent nodes. If odd number of nodes, the last remains in place. Swap nodes, not data values.

```
status_t swap_pairs(list_t* p_list)
```

Sample Input:

[HEAD] ⇄ [1] ⇄ [2] ⇄ [3] ⇄ [4] ⇄ [5] ⇄ ...

Sample Output:

[HEAD] ⇄ [2] ⇄ [1] ⇄ [4] ⇄ [3] ⇄ [5] ⇄ ...

Exercise 14: Split at Middle INTERMEDIATE

Write a function to split the list into two halves. The original list keeps the first half, a new list is returned containing the second half. For odd lengths, first half gets the extra element.

```
list_t* split_list(list_t* p_list)
```

Sample Input:

[HEAD] ⇄ [1] ⇄ [2] ⇄ [3] ⇄ [4] ⇄ [5] ⇄ [6] ⇄ ...

Sample Output:

Original: [HEAD] ⇄ [1] ⇄ [2] ⇄ [3] ⇄ ..., New: [HEAD] ⇄ [4] ⇄ [5] ⇄ [6] ⇄ ...

Exercise 15: Clone List INTERMEDIATE

Write a function that creates and returns a deep copy of the given list. The new list should have completely independent nodes.

```
list_t* clone_list(list_t* p_list)
```

Sample Input:

[HEAD] ⇄ [10] ⇄ [20] ⇄ [30] ⇄ ...

Sample Output:

New independent list: [HEAD] ⇄ [10] ⇄ [20] ⇄ [30] ⇄ ...

Exercise 16: Rotate Left by K INTERMEDIATE

Write a function to rotate the list left by K positions. Handle K greater than list length by using modulo.

```
status_t rotate_left(list_t* p_list, int k)
```

Sample Input:

[HEAD]⇌[1]⇌[2]⇌[3]⇌[4]⇌[5]⇌..., k = 2

Sample Output:

[HEAD]⇌[3]⇌[4]⇌[5]⇌[1]⇌[2]⇌...

Exercise 17: Rotate Right by K INTERMEDIATE

Write a function to rotate the list right by K positions. Handle K greater than list length by using modulo.

```
status_t rotate_right(list_t* p_list, int k)
```

Sample Input:

[HEAD] ⇄ [1] ⇄ [2] ⇄ [3] ⇄ [4] ⇄ [5] ⇄ ..., k = 2

Sample Output:

[HEAD] ⇄ [4] ⇄ [5] ⇄ [1] ⇄ [2] ⇄ [3] ⇄ ...

Exercise 18: Check if Palindrome INTERMEDIATE

Write a function to check if the list is a palindrome. Use the bidirectional nature of DLL: compare from both ends moving towards the middle simultaneously.

```
status_t is_palindrome(list_t* p_list)
```

Sample Input:

[HEAD] ⇄ [1] ⇄ [2] ⇄ [3] ⇄ [2] ⇄ [1] ⇄ ...

Sample Output:

TRUE

Exercise 19: Separate Even and Odd INTERMEDIATE

Write a function to rearrange the list so all even numbers come before all odd numbers. Maintain relative order within even and odd groups.

```
status_t separate_even_odd(list_t* p_list)
```

Sample Input:

[HEAD] ⇄ [1] ⇄ [2] ⇄ [3] ⇄ [4] ⇄ [5] ⇄ [6] ⇄ ...

Sample Output:

[HEAD] ⇄ [2] ⇄ [4] ⇄ [6] ⇄ [1] ⇄ [3] ⇄ [5] ⇄ ...

Exercise 20: Get Union of Two Lists INTERMEDIATE

Write a function that returns a new list containing all unique elements from both lists (no duplicates in result).

```
list_t* get_union(list_t* p_list_1, list_t* p_list_2)
```

Sample Input:

List1: [1]⇒[2]⇒[3], List2: [2]⇒[3]⇒[4]

Sample Output:

New: [HEAD]⇒[1]⇒[2]⇒[3]⇒[4]⇒...

Exercise 21: Get Intersection of Two Lists INTERMEDIATE

Write a function that returns a new list containing only elements present in both lists (no duplicates in result).

```
list_t* get_intersection(list_t* p_list_1, list_t* p_list_2)
```

Sample Input:

List1: [1]⇌[2]⇌[3]⇌[4], List2: [3]⇌[4]⇌[5]

Sample Output:

New: [HEAD]⇌[3]⇌[4]⇌...

Exercise 22: Remove Duplicates from Sorted List INTERMEDIATE

Write a function to remove duplicates from a sorted list. This is more efficient than the unsorted version as duplicates are adjacent.

```
len_t remove_duplicates_sorted(list_t* p_list)
```

Sample Input:

[HEAD]⇌[1]⇌[1]⇌[2]⇌[3]⇌[3]⇌[3]⇌[4]⇌...

Sample Output:

[HEAD]⇌[1]⇌[2]⇌[3]⇌[4]⇌..., returns 3

Section C: Difficult Exercises

Exercise 23: Sort List - Merge Sort **DIFFICULT**

Write a function to sort the list using merge sort algorithm. Maintain correct prev pointers after sorting. Do not convert to array.

```
status_t merge_sort(list_t* p_list)
```

Sample Input:

[HEAD] ⇄ [4] ⇄ [2] ⇄ [1] ⇄ [3] ⇄ ...

Sample Output:

[HEAD] ⇄ [1] ⇄ [2] ⇄ [3] ⇄ [4] ⇄ ...

Exercise 24: Sort List - Insertion Sort **DIFFICULT**

Write a function to sort the list using insertion sort. Take advantage of bidirectional traversal for finding insertion position.

```
status_t insertion_sort(list_t* p_list)
```

Sample Input:

[HEAD] ⇄ [4] ⇄ [2] ⇄ [1] ⇄ [3] ⇄ ...

Sample Output:

[HEAD] ⇄ [1] ⇄ [2] ⇄ [3] ⇄ [4] ⇄ ...

Exercise 25: Reverse in Groups of K **DIFFICULT**

Write a function to reverse nodes in groups of K. If remaining nodes are less than K, reverse them too. Maintain correct prev pointers.

```
status_t reverse_in_groups(list_t* p_list, int k)
```

Sample Input:

[HEAD] ⇄ [1] ⇄ [2] ⇄ [3] ⇄ [4] ⇄ [5] ⇄ [6] ⇄ [7] ⇄ [8] ⇄ ..., k = 3

Sample Output:

[HEAD] ⇄ [3] ⇄ [2] ⇄ [1] ⇄ [6] ⇄ [5] ⇄ [4] ⇄ [8] ⇄ [7] ⇄ ...

Exercise 26: Partition Around Value DIFFICULT

Write a function to partition the list around value X: all nodes less than X come before nodes greater than or equal to X. Preserve original relative order within each partition.

```
status_t partition(list_t* p_list, data_t x)
```

Sample Input:

[HEAD]⇨[1]⇨[4]⇨[3]⇨[2]⇨[5]⇨[2]⇨..., x = 3

Sample Output:

[HEAD]⇨[1]⇨[2]⇨[2]⇨[4]⇨[3]⇨[5]⇨...

Exercise 27: Zigzag/Fold Rearrangement **DIFFICULT**

Write a function to rearrange the list in zigzag pattern: first, last, second, second-last, third, third-last, etc. Use bidirectional traversal capability.

```
status_t zigzag_rearrange(list_t* p_list)
```

Sample Input:

[HEAD]⇌[1]⇌[2]⇌[3]⇌[4]⇌[5]⇌[6]⇌...

Sample Output:

[HEAD]⇌[1]⇌[6]⇌[2]⇌[5]⇌[3]⇌[4]⇌...

Exercise 28: Add Two Numbers as Lists **DIFFICULT**

Two lists represent non-negative integers with digits stored in reverse order (least significant first). Write a function to add them and return the sum as a new list.

```
list_t* add_lists_as_numbers(list_t* p_list_1, list_t* p_list_2)
```

Sample Input:

List1: [2]⇌[4]⇌[3] (=342), List2: [5]⇌[6]⇌[4] (=465)

Sample Output:

New: [HEAD]⇌[7]⇌[0]⇌[8]⇌... (=807)

Exercise 29: Flatten Multi-Level List **DIFFICULT**

Design a multi-level doubly linked list where each node can have a 'child' pointer to another list. Write a function to flatten it into a single-level list. Child lists are inserted after their parent, before the parent's next.

```
status_t flatten_list(multi_list_t* p_list)
```

Sample Input:

1⇌2⇌3⇌4 (2 has child: 5⇌6, 6 has child: 7⇌8)

Sample Output:

[HEAD]⇌[1]⇌[2]⇌[5]⇌[6]⇌[7]⇌[8]⇌[3]⇌[4]⇌...

Exercise 30: LRU Cache Implementation **DIFFICULT**

Implement a Least Recently Used (LRU) cache using your doubly linked list. Most recently used items are at front. When capacity exceeded, remove from end (least recently used).

```
typedef struct lru_cache lru_cache_t;
lru_cache_t* lru_create(int capacity);
int lru_get(lru_cache_t* cache, int key);
void lru_put(lru_cache_t* cache, int key, int value);
```

Sample Operations:

create(2), put(1,1), put(2,2), get(1)→1, put(3,3)→evicts 2, get(2)→-1

Note: For $O(1)$ operations, use a hash table alongside the list. An $O(n)$ search-based solution using only the list is also acceptable.

Summary

Exercise Categories

Traversal & Access: Exercises 1, 2, 6, 7

Positional Operations: Exercises 3, 4, 9

Counting & Comparison: Exercises 5, 8

Modification: Exercises 10, 11, 12, 13, 19, 22, 26

List Operations: Exercises 14, 15, 16, 17, 20, 21

Verification: Exercise 18

Sorting: Exercises 23, 24

Complex Manipulation: Exercises 25, 27, 28, 29

Applications: Exercise 30

Implementation Tips

1. **Use generic_insert and generic_delete:** Build new functions using your existing helpers.
2. **Leverage bidirectional traversal:** For palindrome check, zigzag, etc., use both forward and backward traversal.
3. **Handle edge cases:** Empty list, single element, two elements, odd/even lengths.
4. **Maintain prev pointers:** After any manipulation, ensure all prev pointers are correct.
5. **DLL vs DCLL adaptation:** For DLL, check `p_run != NULL`. For DCLL, check `p_run != p_list`.

Testing Recommendations

Create test functions in `use_list.c` following the existing pattern:

```
assert(function_name(p_list, args) == EXPECTED_RESULT);  
show_list(p_list, "After operation:");
```

Test with: empty list, single element, two elements, sorted input, reverse sorted input, all same values, alternating values.